

# Project Proposal: High-Performance N-Body Gravity Simulation Using CUDA

**Team Members:** Ali Amini, Abolfazl Najm , **Student Number:** 40231990, 40231967

**Topic:** Topic 2 — N-Body Simulation (Solar System & Comet)

**Course:** Multicore and GPU Programming

**Hardware Target:** NVIDIA GeForce RTX 3050 Laptop GPU (Ampere Architecture)

## 1. Abstract

We propose to implement a high-performance N-Body simulation to model the gravitational evolution of a solar system consisting of a central star, planets, and comets. The simulation will compute pairwise interactions between all bodies with a computational complexity of  $O(N^2)$ . The primary objective is to maximize simulation throughput (interactions per second) by implementing a Shared Memory Tiling strategy to overcome the global memory bandwidth bottlenecks inherent in this problem. The project will feature a real-time visualization to verify physical correctness, such as stable orbits and comet deflection. Per the project requirements, we will also implement a real-time visualization to verify physical correctness.

## 2. Problem Description & Physical Model

We will model a system of  $N$  bodies where everybody interacts with every other body.

- **Interaction Rule:** We will use Newton's Law of Universal Gravitation. For every pair of bodies  $(i, j)$ , the force exerted is proportional to  $\frac{m_i}{r^2} m_j$
- **System Scope:** The simulation will include a massive central body (Sun), orbiting planets, and a comet with a highly elliptical orbit to demonstrate gravitational deflection.
- **Complexity:** As the number of bodies  $N$  increases, the number of force calculations grows quadratically ( $N^2$ ). For large  $N$ , this becomes computationally expensive, making it an ideal candidate for massive parallelization on the GPU.

## 3. Technical Approach & Optimization Strategy

The project treats this N-Body simulation as a **parallel numerical simulation problem**. We will focus on the following CUDA optimizations, tailored for the RTX 3050:

### A. Parallelization Strategy

We will map the problem such that **one CUDA thread computes the total force for one body**

- **Grid Configuration:** We will use 1D thread blocks (e.g., 128 or 256 threads per block) to align with the linear memory layout of the particle data.

## B. Memory Optimization (The Core Challenge)

A naive implementation requires  $2N$  global memory reads for every thread, which will saturate the memory bandwidth of the RTX 3050 Laptop GPU. To address this, we will implement **Shared Memory Tiling**:

1. Threads in a block will cooperatively load a "tile" of body positions into on-chip Shared Memory.
2. All threads in the block will compute interactions with these cached bodies.
3. The block moves to the next tile. This technique significantly reduces global memory traffic, which is critical for scaling performance.

## C. Data Structure

We will use a structure-of-arrays (SoA) or `float4` vectors (position.x, position.y, position.z, mass) to ensure **coalesced memory access** when reading data from global memory, maximizing bandwidth utilization.

# 4. Visualization

To verify the physical behavior (e.g., comet deflection), we will implement a real-time visual layer using **OpenGL interoperability** with CUDA. This allows us to render the system state directly from GPU memory without expensive CPU-GPU data transfers, ensuring the simulation remains the performance bottleneck, not the visualization.

# 5. Workload Distribution (Team of 2)

We will divide the work to ensure parallel development:

- **Member 1 (Kernel Architect):**
  - Implementation of the CUDA Kernels (Integrator and Force Calculation).
  - Implementation of Shared Memory Tiling optimization.
  - Performance profiling (measuring kernel execution time).
- **Member 2 (Host & Visualization):**
  - CPU-side data initialization (setting up the Solar System initial states).
  - OpenGL/Graphics setup for the real-time visualization.
  - Managing the main simulation loop and time-stepping logic.

## 6. Evaluation Plan

We will evaluate the project based on:

**Correctness:** Visual confirmation of stable orbits and comet trajectories.

**Scalability:** We will measure the **Interactions Per Second** and time-per-step as N increases.

**Optimization Success:** We will compare the performance of our optimized "Tiled" kernel against a naive "Global Memory" baseline to quantify the speedup achieved.

## 7. Timeline

**Week 1:** Setup build environment; implement CPU baseline and Naive CUDA kernel.

**Week 2:** Implement Shared Memory Tiling optimization; set up OpenGL visualization.

**Week 3:** Refine physics parameters for stable orbits; profile and tune block sizes for RTX 3050.

**Week 4:** Final scaling analysis, performance metrics collection, and report writing.